

Algorithms and Complexity II

Contents

1	Dijkstra's Algorithm	2
1.1	Adjacency matrix	2
1.2	Shortest path	2
1.3	Single-source shortest paths problem	2
1.4	Relaxing an edge	2
1.5	Dijkstra's algorithm	2
1.6	Properties of shortest paths and relaxation	2
2	Flow networks	3
2.1	Maximum-flow problem	3
2.1.1	Flow	3
2.1.2	Implicit summation	4
2.2	The Ford-Fulkerson method	4
2.2.1	Residual networks	4
2.2.2	Augmenting path	4
2.2.3	Cuts	5
2.2.4	Basic Ford-Fulkerson algorithm	5
3	Decision problems	6
3.1	Defining a decision problem	6
3.1.1	Reachability	6
3.1.2	Encoding scheme	6
3.2	Languages	7
3.3	Complexity measures	7
3.3.1	Dynamic complexity measures	7
3.4	Independent Set and Vertex Cover	7
3.5	Clique	8
4	The complexity class $\mathbf{P}$	9
4.1	Time complexity	9
4.2	Time complexity classes	9
4.3	The class $\mathbf{P}$	9
4.3.1	Different models of computation	10
4.3.2	Encodings	10
4.4	Proving a problem is in $\mathbf{P}$	10
4.5	CNF	10

4.5.1	Satisfiability	11
4.6	Polynomial-time reduction	12
4.6.1	k -colourability	12
4.6.2	Reducing 2-Colourability to 2-Satisfiability	13
5	The complexity class NP	13
5.1	Certificate	13
5.2	Verifiers	13
5.3	The class NP	14
5.4	Non-deterministic machines	14
5.4.1	Non-deterministic TM time complexity	15
5.4.2	The class $\text{NTIME}(f)$	15
5.5	Alternative definition of NP	15
5.6	P = NP	15
5.7	Complete problems	16
5.7.1	Ordering by reduction	16
5.7.2	Complete problems	16
5.8	NP-completeness	16
5.8.1	NP-completeness of Satisfiability	17
5.9	Ladner's theorem	17
5.10	NP-intermediate candidates	17
5.11	NP-hard	17
5.12	Proving reducibility	18
5.13	NP -completeness of Vertex Cover	18
5.14	NP-completeness of Clique	19
5.15	NP -completeness of Hitting Set	20
5.15.1	NP -completeness of 3-SAT	20
5.16	Reducing SAT to Clique	20

1 Dijkstra's Algorithm

1.1 Adjacency matrix

An adjacency matrix can be used to represent a graph with edge weights: - a square matrix is used
- rows and columns are labelled with the vertices

The entry in row i and column j is: - w if there is an edge of weight w from i to j , - 0 if there is no edge.

1.2 Shortest path

The weight (or length) of a path from a vertex u to a vertex v is the sum of the weights of the edges in the path.

Note that a weight may be negative.

The shortest path (i.e. with least weight) between u and v is denoted $\delta(u, v)$. If there is no path, then $\delta(u, v) = \infty$.

1.3 Single-source shortest paths problem

The single-source shortest paths problem involves finding the shortest path from a specific source vertex s to all other vertices in the graph.

1.4 Relaxing an edge

Relaxing an edge means checking whether going through that edge gives a shorter path to a vertex, and updating the distance if it does.

1.5 Dijkstra's algorithm

1. Initialisation
 - the node A is the source node
 - the set S stores the vertices v for which $\delta(A, v)$ has been found (initially empty)
 - the set Q stores all the other vertices (initially contains all vertices)
 - $d(v)$ is an estimate of $\delta(A, v)$
 - $d(A) = 0$ (initially infinite for all other vertices)
2. While Q is not empty:
 - remove from Q the vertex u for which $d(u)$ is minimum
 - add this vertex u to S
 - relax all edges leaving u

The running time of Dijkstra's algorithm is $O(V^2)$.

1.6 Properties of shortest paths and relaxation

- Triangle Inequality
 - For all edges (u, v) , we have $\delta(s, v) \leq \delta(s, u) + w(u, v)$.
- Optimal Substructure
 - Any subpath of a shortest path is also a shortest path.
- Upper Bound Property

- For every vertex v , we always have $d(v) \geq \delta(s, v)$.
- No-path Property
 - If $\delta(s, v) = \infty$, then we have $d(v) = \infty$ at every iteration.
- Convergence Property
 - If there is a shortest path from s to v including the edge (u, v) and if $d(u) = \delta(s, u)$, then we obtain $d(v) = \delta(s, v)$ when (u, v) is relaxed.

2 Flow networks

Take a scenario where: - material is transferred in a network from a “source” to a “sink” - source produces material at a steady rate, sink consumes at same rate

For example, water through a system of pipes or current through an electrical network.

Edges have a given capacity - the maximum material transfer per unit of time

Vertices (other than source/sink) are “junctions”: - material flows through them without collecting in them - entering rate = exiting rate (flow conservation)

2.1 Maximum-flow problem

The maximum-flow problem involves computing the greatest possible rate of transportation from source to sink in a flow network.

A flow network is defined by a directed graph $G = (V, E)$, with: - two distinguished vertices: source s and sink t - each edge $(u, v) \in E$ has non-negative capacity $c(u, v) \geq 0$ - if $(u, v) \notin E$, we assume $c(u, v) = 0$.

We assume that every vertex lies on some path from s to t : - for each $v \in V$, there is a path $s \rightsquigarrow v \rightsquigarrow t$ - therefore, G is connected, so $|E| \geq |V| - 1$

2.1.1 Flow

A flow in G is a real-valued function $f : V \times V \rightarrow \mathbb{R}$ that satisfies three properties: - Capacity constraint: - For all $u, v \in V$, we require $f(u, v) \leq c(u, v)$ - Flow from one vertex to another must not exceed given capacity. - Skew symmetry: - For all $u, v \in V$, we require $f(u, v) = -f(v, u)$ - Flow from vertex u to vertex v is negative of flow in reverse direction. - Flow conservation: For all $u \in V - s, t$, we require - $\sum_{v \in V} f(u, v) = 0$ - Total flow out of a vertex (other than source or sink) is 0.

$f(u, v)$ is called the flow from u to v . It can be positive, zero, or negative.

Total flow The total positive flow entering vertex v is

$$\sum_{u \in V: f(u, v) > 0} f(u, v)$$

The total positive flow leaving vertex u is

$$\sum_{v \in V: f(u, v) > 0} f(u, v)$$

The total net flow at a vertex v is: - total positive flow leaving v - total positive flow entering v

Total net flow is: - positive at source - negative at sink - 0 at all other vertices

Flow value The value of flow f is defined as the total flow leaving the source (and thus entering the sink):

$$|f| = \sum_{v \in V} f(s, v)$$

2.1.2 Implicit summation

Let $X, Y \subseteq V$. Then:

$$f(X, Y) = \sum_{x \in X} \sum_{y \in Y} f(x, y)$$

Commonly occurring identities: - For all $X \subseteq V$, we have $f(X, X) = 0$ - Because each $f(u, v)$ and $f(v, u) = -f(u, v)$ cancel each other. - For all $X, Y \subseteq V$, we have $f(X, Y) = -f(Y, X)$ - Generalization of $f(X, X) = 0$, same reasoning. - For all $X, Y, Z \subseteq V$ with $X \cap Y = \emptyset$, we have - $f(X \cup Y, Z) = f(X, Z) + f(Y, Z)$ - $f(Z, X \cup Y) = f(Z, X) + f(Z, Y)$ - Split summation into two: one over X , one over Y

2.2 The Ford-Fulkerson method

The Ford-Fulkerson method is an iterative method for solving the maximum flow problem: 1. Start with $f(u, v) = 0$ for all $u, v \in V$ 2. At each iteration, increase flow value by finding an augmenting path — a path from source to sink along which we can increase flow — and then augment flow along this path. 3. Repeat until no augmenting path can be found

2.2.1 Residual networks

The residual network consists of edges that can admit more flow.

Consider vertices u and v . The amount of additional flow we can push from u to v before exceeding capacity $c(u, v)$ is the residual capacity of (u, v) : - $c_f(u, v) = c(u, v) - f(u, v)$

Note: When flow $f(u, v)$ is negative, then residual capacity $c_f(u, v)$ is greater than $c(u, v)$.

Given flow network $G = (V, E)$ and flow f , the residual network of G induced by f is $G_f = (V, E_f)$ with

$$E_f = \{(u, v) \in V \times V : c_f(u, v) > 0\}$$

i.e. each residual edge that can admit flow that is strictly positive.

Let $G = (V, E)$ be a flow network, f be a flow in G , G_f be the residual network of G induced by f , and let f' be a flow in G_f . Then the flow sum $f + f'$ with

$$(f + f')(u, v) = f(u, v) + f'(u, v)$$

is a flow in G with value $|f + f'| = |f| + |f'|$.

2.2.2 Augmenting path

An augmenting path is a path in the residual graph from the source s to the sink t where every edge has residual capacity > 0 .

Along this path, you can increase the flow by the minimum residual capacity on the path.

2.2.3 Cuts

A cut (S, T) is a partition of all the nodes V into two sets: - S — contains the source s - T — contains the sink t

Every node is in exactly one of the two sets: $S \cup T = V$, $S \cap T = \emptyset$

If f is a flow in G , then $f(S, T)$ is the net flow across cut (S, T) ; its capacity is $c(S, T)$. - net flow includes negative flows across the cut; capacity includes only non-negative values

The net flow across any cut is the same. - since flow is conserved at every node apart from s and t

A minimum cut is a cut with minimum capacity over all cuts.

The value of the flow f in a network G is upper bounded by the capacity of any cut (S, T) in G .

Max-flow min-cut If f is a flow in a flow network $G = (V, E)$ with source s and sink t , then the following conditions are equivalent: 1. f is a maximum flow in G . 2. The residual network G_f contains no augmenting path. 3. $|f| = c(S, T)$ for some cut (S, T) of G .

2.2.4 Basic Ford-Fulkerson algorithm

Input: G, s, t

```
for each edge  $(u, v)$  in  $E$  do
     $f(u, v) \leftarrow 0$ 
     $f(v, u) \leftarrow 0$ 
end for
```

```
while there exists a path  $P = s \rightarrow t$  in the residual network  $G_f$  do
    {i.e. augmenting path in  $G$ }
     $c_f(P) \leftarrow \min\{c_f(u, v) : (u, v) \text{ in } P\}$ 

    for each edge  $(u, v)$  in  $P$  do
         $f(u, v) \leftarrow f(u, v) + c_f(P)$ 
         $f(v, u) \leftarrow -f(u, v)$ 
    end for
end while
```

- Start with zero flow
 - Set flow on every edge to 0
- While there is a path from s to t (augmenting path)
 - Find a path where more flow can be sent (in the residual graph)
 - Find the bottleneck capacity
 - * Look at all edges in the path
 - * Take the minimum remaining capacity
 - Push flow along the path
 - * Add the bottleneck value to each forward edge
 - * Update backward edges to allow flow to be undone if needed
- Repeat
 - Keep finding new paths and pushing flow
- Stop when no augmenting path exists

- No more flow can be sent from s to t
- Result
 - The total flow is the maximum flow

Running time The running time of the algorithm is highly dependent on how augmenting paths are determined for the while loop.

If chosen poorly: - value of flow increases with each iteration - but possibly too slowly - in extreme cases it might never terminate, and it can even not converge to the value of maximum flow (this can happen only if the capacities are irrational numbers)

3 Decision problems

A decision problem is a problem whose answer is simply yes or no.

An optimisation problem is a problem that asks for the best solution among all feasible solutions.

Every optimisation problem has a decision problem counterpart:

- For example, the shortest path problem:
 - optimisation: given vertices u and v , what is the shortest path from u to v .
 - decision: given vertices u, v and value k does there exist a path from u to v with length at most k ?

An optimisation problem has a polynomial algorithm if and only if the corresponding decision problem has a polynomial algorithm.

3.1 Defining a decision problem

A decision problem is usually defined by:

- describing a generic instance of the problem
- asking a yes/no question about that instance

3.1.1 Reachability

The reachability problem is an example of a decision problem:

- Given a finite directed graph G and vertices s and t , is there a path in G from s to t

3.1.2 Encoding scheme

In order to input a problem to a computer, each problem instance must be encoded as a string of symbols over some alphabet (typically binary).

Therefore, an encoding scheme is needed.

This encoding scheme must be concise and efficient:

- numbers should be represented efficiently (e.g. binary rather than unary/base 1)
- unnecessary or redundant information should be avoided

This is important because the running time of algorithms is measured in terms of the input size. An inefficient encoding could artificially change the apparent complexity of a problem.

In complexity theory, reasonable encoding schemes differ only polynomially in size, so they do not change the fundamental classification of a problem (for example, whether a problem is in P or NP).

3.2 Languages

An alphabet Σ is a finite set of symbols - e.g., $\Sigma = 0, 1$

A string over Σ is a finite sequence of symbols from Σ - e.g., 01101

A language over Σ is any set of strings over Σ .

For a decision problem Π , and an encoding scheme e over some alphabet Σ

$$L(\Pi, e)$$

is defined to be the set of all encoded instances for which the answer to Π is yes.

This set is called the language associated with Π and e .

Thus, solving a decision problem is equivalent to deciding whether a given encoded instance belongs to the language $L(\Pi, e)$.

3.3 Complexity measures

Every decidable problem has at least one algorithm that solves it.

To classify problems, we study properties of the set of all algorithms that solve the problem.

A key idea is to consider the efficiency of the best possible algorithm in this set, measured in terms of resource usage. This is a dynamic complexity measure.

A complexity measure is a function that quantifies the amount of computational resources used by an algorithm as a function of the input.

3.3.1 Dynamic complexity measures

The most important computational resources are usually time and space.

Using Turing Machines as the model of computation:

- The time complexity of a Turing Machine T is the function $Time_T(x)$ which is the number of computation steps taken by T on input x .
- The space complexity of a Turing Machine T is the function $Space_T(x)$ which is the number of tape cells visited during the computation of T on input x .

If $T(x)$ does not halt, then both $Time_T(x)$ and $Space_T(x)$ are undefined.

3.4 Independent Set and Vertex Cover

The independent set problem is defined as:

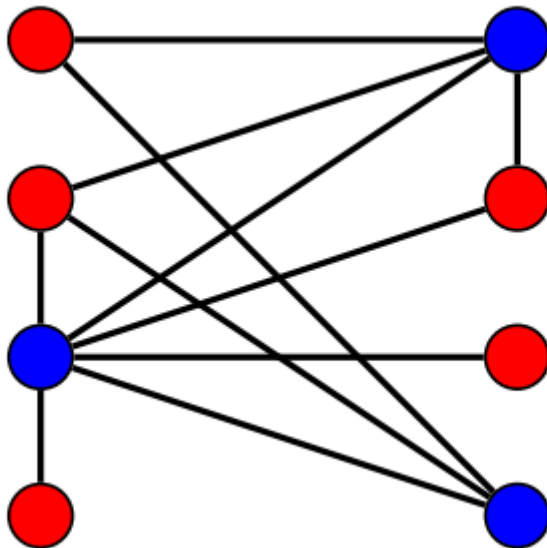
- Given a graph $G = (V, E)$ and an integer k ,
- does G contain an independent set of size at least k ?

An independent set is a set of vertices such that no two vertices in the set are joined by an edge.

The vertex cover problem is defined as:

- Given a graph $G = (V, E)$ and an integer k ,
- does G contain a vertex cover of size at most k ?

A vertex cover is a set of vertices such that every edge in the graph has at least one endpoint in the set.



In this example graph:

- there is an independent set of size 5 (the red vertices)
- there is a vertex cover of size 3 (the blue vertices)

For a graph G with n vertices:

- G has an independent set of size k if and only if G has a vertex cover of size $n - k$

Therefore, the independent set and vertex cover decision problems are polynomial-time equivalent (they can be transformed into each other in polynomial time via the above relationship).

3.5 Clique

The clique problem is defined as:

- Given a graph $G = (V, E)$ and an integer k ,
- does G contain a clique of size at least k ?

A clique is a set of vertices in which every pair of distinct vertices is connected by an edge.

The complement of a graph $G = (V, E)$, denoted $\overline{G}$, is the graph with:

- the same vertex set V
- an edge between two vertices u and v in G if and only if $(u, v) \notin E$

A graph G has an independent set of size k if and only if its complement $\overline{G}$ has a clique of size k .

Therefore, the Clique problem and the Independent Set problem are polynomial-time equivalent, since one can be transformed into the other via taking graph complements in polynomial time.

4 The complexity class $\mathbf{P}$

4.1 Time complexity

For any function f , a decidable language L has time complexity $O(f)$ if there exist:

- a deterministic Turing machine T that decides L , and
- constants $c > 0$ and $n_0 \geq 0$,

such that for every input x with $|x| \geq n_0$,

$$\text{Time}_T(X) \leq c \cdot f(|x|)$$

Here:

- $|x|$ denotes the length of the input,
- $\text{Time}_T(x)$ is the number of computation steps taken by T on input x .

4.2 Time complexity classes

For a function f , the complexity class

$$\text{TIME}(f)$$

(or sometimes $\text{DTIME}(f)$, for deterministic time) is the set of all languages L for which there exists a deterministic Turing machine that decides L in $O(f)$ time.

4.3 The class $\mathbf{P}$

The complexity class $\mathbf{P}$ is defined by:

$$\mathbf{P} = \bigcup_{k \geq 0} \text{TIME}[n^k]$$

This is the class of all languages that can be decided in polynomial time by a deterministic Turing machine.

The class $\mathbf{P}$ is commonly considered a mathematical model of tractable (efficiently solvable) problems.

However, the correspondence with “practical efficiency” is only approximate:

- A polynomial-time algorithm with a large exponent may still be impractical for real input sizes
- The hidden constant factors in the $O()$ notation may also be very large and dominate runtime in practice

Generally for naturally occurring problems in computer science, the degree and constants are not too large.

4.3.1 Different models of computation

- A k -tape Turing machine that runs in t steps can be simulated by an equivalent single-tape Turing machine in $O(t^2)$ time.
- A single-tape Turing machine running in t steps can be simulated by a k -tape Turing machine in $O(t)$ time.

Therefore, the complexity class **P** is invariant under these standard models of computation (up to polynomial factors), including single-tape, multi-tape, and other reasonable deterministic machine models.

4.3.2 Encodings

For any integer n , the length of its representation in base b_1 base b_2 differ by at most a constant factor, provided $b_1, b_2 \geq 2$.

For any graph G , the length of the encoding of G :

- as an adjacency matrix, and
- as a list of edges

differ in size by at most a polynomial factor in the number of vertices.

Hence the class **P** is invariant under these reasonable encodings of inputs (up to polynomial transformations of input length).

4.4 Proving a problem is in **P**

The class **P** is considered robust:

- it is invariant under reasonable choices of computational model (e.g. single-tape vs multi-tape Turing machines)
- it is also invariant under reasonable encodings of inputs

Therefore, when proving that a problem is in **P**, the exact machine model or low-level encoding details are usually not important.

To show that a problem (language) is in **P**, the standard approach is:

- Provide a deterministic algorithm that solves the problem in polynomial time

This algorithm can be described at a high level (e.g. in pseudo-code), as long as its correctness and polynomial running time can be justified:

- even a straightforward polynomial-time algorithm is useful, because it establishes that the problem is efficiently solvable in theory
- however, finding such an algorithm often requires identifying a strategy that is significantly better than brute-force search

4.5 CNF

Many important computational problems involve logical formulas over Boolean variables.

A Boolean formula f is in conjunctive normal form (CNF) if it can be written as:

$$f = C_1 \wedge \dots \wedge C_m$$

where each C_i is a clause.

A clause is a disjunction (OR) of literals:

$$C_i = L_{i1} \vee \dots \vee L_{ik}$$

A literal is either: - a variable, e.g., x - or its negation, e.g., $\neg x$

A formula is in k -CNF if every clause contains at most k literals.

4.5.1 Satisfiability

A Boolean formula f in CNF is satisfiable if there exists an assignment of TRUE / FALSE values to its variables such that the formula evaluates to TRUE.

- f is TRUE iff all clauses are TRUE
- a clause is TRUE iff at least one of its literals is TRUE

The Satisfiability problem The Satisfiability problem (SAT) is:

Given a CNF formula f , does there exist a truth assignment to its variables such that f evaluates to TRUE?

k -Satisfiability problem A restricted version is the k -SAT problem:

Given a CNF formula in which every clause has at most k literals, is the formula satisfiable?

2-Satisfiability 2-Satisfiability is in **P**.

The proof is as follows:

0. Declare:
 - all clauses unsatisfied
 - all literals unassigned
1. Select an arbitrary unassigned variable x . Assign:
 - $x = \text{TRUE}$
 - $\neg x = \text{FALSE}$
2. Select an unsatisfied clause $I_i \vee I_j$:
 - if both literals are unassigned:
 - ignore the clause
 - if at least one literal is assigned TRUE:
 - declare the clause satisfied
 - if one literal is FALSE and the other is unassigned:
 - assign the unassigned literal to TRUE
 - assign its negation to FALSE
 - mark the clause as satisfied
 - if both literals are FALSE:
 - restart the algorithm
 - set $x = \text{FALSE}$ and $\neg x = \text{TRUE}$
 - if the same conflict occurs again after this restart, declare the formula unsatisfiable
3. Repeat step 2 until either:

- all clauses are satisfied:
 - return satisfiable
- remaining unsatisfied clauses contain only unassigned variables:
 - return to step 1

4.6 Polynomial-time reduction

An alternative way to prove that a problem is in **P** is by using a reduction.

Informally, a problem P is reducible to a problem Q if an algorithm for solving Q can be used as a subroutine to solve P .

Formally: a language L_1 is polynomially reducible to L_2 , denoted

$$L_1 \leq L_2$$

if there exists a function f such that:

- f is computable in polynomial time, and
- for all inputs x :

$$x \in L_1 \iff f(x) \in L_2$$

The function f transforms instances of problem L_1 into instances of problem L_2 such that:

- YES-instances stay YES-instances
- NO-instances stay NO-instances

So solving L_2 automatically gives a way to solve L_1 .

The key property of this is that:

- if $L_1 \leq L_2$ and $L_2 \in \mathbf{P}$, then $L_1 \in \mathbf{P}$

4.6.1 k -colourability

A (vertex) colouring of a graph $G = (V, E)$ is a function:

$$f : V \rightarrow 1, \dots, k$$

A colouring is proper if for every edge $(u, v) \in E$,

$$f(u) \neq f(v)$$

That is, adjacent vertices must receive different colours.

The k -Colourability problem is:

Given a graph G , does there exist a proper colouring of G using at most k colours?

Note that when $k = 2$, this problem becomes exactly the bipartite graph problem.

Given a graph G , can the vertices be split into two disjoint sets, such that no vertex shares an edge with any other vertex in its set?

4.6.2 Reducing 2-Colourability to 2-Satisfiability

The 2-Colourability (bipartite) problem can be reduced to 2-SAT:

- for each vertex v_i of the graph
 - create a variable x_i
- for each edge (v_i, v_j) :
 - add two clauses:
 - * $(x_i \vee x_j)$
 - * $(\neg x_i \vee \neg x_j)$

This converts the 2-Colourability problem to the 2-Satisfiability problem in $O(V + E)$.

Solving the 2-Colourability problem can now be done by solving the equivalent 2-Satisfiability problem:

- if the formula is satisfiable, define the 2-colouring by setting TRUE variables to colour 1 and FALSE to colour 2
- if two adjacent vertices get the same colour, then the corresponding edge clauses are not satisfied (contradiction)
- thus we obtain a proper 2-colouring

5 The complexity class NP

5.1 Certificate

A certificate (or witness) is a string that provides evidence that an input belongs to a language.

More formally, for a language L , a certificate for an input x is a string c that can be used by a polynomial-time verification algorithm to confirm that:

$$x \in L$$

- A certificate is often thought of as a proposed solution that can be checked efficiently.
- For inputs whose answer is YES, there exists a certificate that verifies this.
- For inputs whose answer is NO, no valid certificate exists.

For example:

- Satisfiability (SAT):
 - a certificate is a satisfying assignment of truth values to the variables
- k -Colourability:
 - a certificate is a proper k -colouring of the graph
- Hamiltonian Cycle:
 - a certificate is a Hamiltonian cycle in the graph

5.2 Verifiers

An acceptor machine V that halts on all inputs is called a verifier for a language L if

$$L = \{w \mid V \text{ accepts } \langle w, c \rangle \text{ for some string } c\}$$

- the string c is called a certificate (or witness) for w
- the verifier receives both the input w and the certificate c
- if $w \in L$, then there exists a certificate c that causes V to accept
- If $w \notin L$, then no certificate causes V to accept

A verifier is said to be polynomial-time if:

- V runs in polynomial time with respect to the size of its input, and
- there exists a polynomial p such that, for every $w \in L$, there is a certificate c satisfying

$$|c| \leq p(|w|)$$

That is, every YES-instance has a certificate whose length is polynomial in the size of the input.

5.3 The class NP

The class **NP** is the set of all languages that have a polynomial-time verifier.

That is, a language is in **NP** if every YES-instance has a polynomial-size certificate that can be verified in polynomial time.

Examples of problems in **NP** include:

Composite Number > Given a positive integer k , are there integers $u, v > 1$ such that $u \cdot v = k$

- A certificate is the pair (u, v) .
- Verification consists of checking that:
 - $u, v > 1$, and
 - $u \cdot v = k$
- Both checks can be performed in polynomial time.

Subset Sum > Given a collection of positive integers $S = a_1, \dots, a_k$ and a target integer t , is there a subset $T \subseteq S$ whose elements sum to t ?

- A certificate is the subset T .
- Verification consists of summing the elements of T and checking whether the result is t .
- This can be done in polynomial time.

Some problems that are probably not in **NP** include:

No Hamiltonian Cycle > Given a graph G , is it true that G has no Hamiltonian cycle?

- a Hamiltonian cycle itself is a certificate that a graph does have a Hamiltonian cycle
- However, it is not known how to provide a polynomial-size certificate that proves a graph has no Hamiltonian cycle.

5.4 Non-deterministic machines

An alternative definition of the class **NP** is given using non-deterministic Turing machines.

A non-deterministic Turing machine NT can be viewed as a computation tree:

- $NT(x)$ denotes the tree of all possible computation paths on input x
- Each node is a configuration, and each branch represents a possible nondeterministic choice

- NT accepts x if at least one computation path ends in an accept state

5.4.1 Non-deterministic TM time complexity

Let NT be a non-deterministic Turing machine.

The time complexity of NT on input x , denoted:

$$\text{NTime}_{NT}(x)$$

is defined as the number of steps in the shortest accepting path of $NT(x)$ if there is one, otherwise it is the number of steps in the shortest rejecting path.

If not all paths of $NT(x)$ halt, then $\text{NTime}_{NT}(x)$ is undefined.

For a function f , we say a language L is decidable in non-deterministic time $O(f)$ if

- there exists a non-deterministic Turing machine NT such that:
 - NT decides L
 - and there exist constants $c > 0$, n_0 such that for all inputs x with $|x| > n$, every computation path of $NT(x)$ halts within:

$$\text{NTime}_{NT}(x) \leq c \cdot f(|x|)$$

5.4.2 The class $\text{NTIME}(f)$

The class

$$\text{NTIME}(f)$$

is the set of all languages that can be decided by a nondeterministic Turing machine in time $O(f)$.

5.5 Alternative definition of NP

Using non-deterministic machines, NP can be defined as:

$$\text{NP} = \bigcup_{k \geq 0} \text{NTIME}[n^k]$$

This is the class of all languages that can be decided by a nondeterministic Turing machine in polynomial time.

5.6 $\text{P} = \text{NP}$

The question of whether $\text{P} = \text{NP}$ is one of the most important open problems in computer science.

It asks whether every problem whose solution can be verified efficiently can also be solved efficiently.

Equivalently, does the existence of a short (polynomial-size) certificate guarantee the existence of a polynomial-time algorithm to find such a certificate?

5.7 Complete problems

Any complexity class can be structured using polynomial-time reductions.

- We say L_1 is related to L_2 if $L_1 \leq L_2$
- This induces a notion of relative difficulty between problems

This allows us to group problems into equivalence classes:

- Two problems are in the same equivalence class if they are polynomial-time reducible to each other

$$L_1 \leq L_2 \text{ and } L_2 \leq L_1$$

- Such problems are considered equally hard (up to polynomial time)

5.7.1 Ordering by reduction

Polynomial-time reductions define a preorder on problems:

- if $L_1 \leq L_2$, then L_1 is no harder than L_2

This induces a partial ordering on equivalence classes of problems.

5.7.2 Complete problems

A problem L in a complexity class C is called complete for C if:

- $L \in C$, and
- every problem in C reduces to L in polynomial time

$$\forall L' \in C, \quad L' \leq L$$

Complete problems are the hardest problems in the class (up to polynomial-time reductions): - If we can solve one complete problem efficiently, then we can solve all problems in the class efficiently

5.8 NP-completeness

NP-complete problems are the hardest problems in NP.

They have the property that:

- they are in NP, and
- every problem in NP can be reduced to them in polynomial time

This means:

- If any NP-complete problem can be solved in polynomial time, then every problem in NP can be solved in polynomial time.

To show that L is NP-complete, it must be shown that every language in NP can be reduced to L in polynomial time:

- once one NP-complete language L_0 is known, any language L can be shown to be NP-complete by showing that $L_0 \leq L$ and $L \in NP$.

5.8.1 NP-completeness of Satisfiability

The Cook-Levin theorem is that $\geq$ Satisfiability is NP-complete

so:

- SAT is in NP, and
- every problem in NP can be reduced to SAT in polynomial time

This would mean that:

- $\mathbf{P} = \mathbf{NP}$ if and only if Satisfiability is in $\mathbf{P}$.

5.9 Ladner's theorem

Ladner's theorem states that:

- if $\mathbf{P} \neq \mathbf{NP}$, then there exist problems in NP that are neither in $\mathbf{P}$ nor NP-complete.

Such problems are called NP-intermediate problems.

This means that, if $\mathbf{P} \neq \mathbf{NP}$, then:

- $\mathbf{NP}$ is strictly larger than $\mathbf{P}$
- not all problems in $\mathbf{NP} \setminus \mathbf{P}$ are NP-complete
- there exist middle problems of intermediate difficulty

5.10 NP-intermediate candidates

Proving that a problem is NP-intermediate would imply that $\mathbf{P} \neq \mathbf{NP}$. Therefore, no problem has been proven to be NP-intermediate.

However, there are several candidates that are believed to lie in NP-intermediate.

A leading candidate is:

Graph Isomorphism

Given two graphs $G = (V_G, E_G)$ and $H = (V_H, E_H)$, are G and H isomorphic?

Two graphs are isomorphic if there is a bijection

$$f : V_G \rightarrow V_H$$

such that

$$(u, v) \in E_G \iff (f(u), f(v)) \in E_H$$

5.11 NP-hard

To prove a problem Π is **NP**-complete, two steps must be performed:

1. Show that $\Pi \in \mathbf{NP}$
2. Find a known **NP**-complete problem Π' and show that $\Pi' \leq \Pi$

If step 2 can be shown, but step 1 is not known or does not hold, then Π is **NP**-hard.

5.12 Proving reducibility

There are three general methods to prove that a known NP-complete problem Π' is polynomial-time reducible to a problem Π :

- restriction
 - show that Π' is a special case (restriction/subproblem) of Π
 - That is, every instance of Π' is also an instance of Π
- local replacement
 - show that every basic unit (e.g. variable, clause, edge) in an instance of Π' can be replaced by a corresponding structure in Π
 - the replacement must be uniform and preserve correctness
- component design
 - show that instances of Π can be used to construct gadgets (components) that simulate the behaviour of Π'
 - these gadgets are then combined to encode any instance of Π'

5.13 NP-completeness of Vertex Cover

Recall the vertex cover problem:

Given a graph $G = (V, E)$ and an integer k , does G contain a vertex cover of size at most k ?

A vertex cover is a set of vertices such that every edge in the graph has at least one endpoint in the set.

This problem is in **NP**:

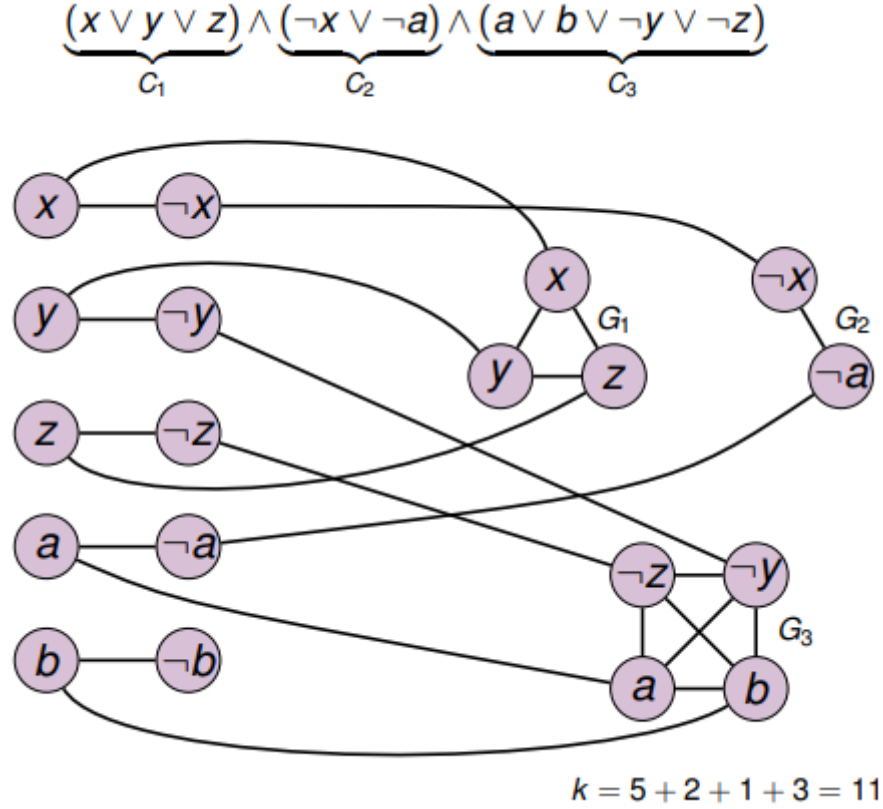
- a certificate is a proposed vertex cover

To show that Vertex Cover is NP-complete, Satisfiability (known to be **NP**-complete) can be reduced to Vertex Cover:

Given a CNF formula f with n variables and clauses $C_1, C_2, \dots, C_m$:

- for each variable x :
 - create two adjacent vertices x^t and x^f
 - these represent the literals x and $\neg x$
- for each clause C_j of size n_j :
 - create a complete graph (clique) G_j on n_j vertices
 - each vertex corresponds to one literal in the clause
- for each literal vertex in a clause gadget:
 - connect it to the corresponding literal vertex in the variable gadget
- set $k = n + \sum_{j=1}^m (n_j - 1)$

This construction can be carried out in polynomial time.



Then, exists a truth assignment that satisfies the formula f if and only if there exists a vertex cover of the constructed graph with size at most k .

Therefore, since SAT is NP-complete, it follows that vertex cover is NP-complete.

5.14 NP-completeness of Clique

Recall the clique problem:

Given a finite graph G and an integer k , does G have a clique of size k ?

A clique is a set of vertices such that every pair of distinct vertices in the set is connected by an edge.

There is a straightforward reduction from Vertex Cover to Clique:

Let $\overline{G}$ denote the complement of G , i.e. the graph obtained by:

- keeping the same vertices, and
- replacing every non-edge of G with an edge, and vice versa

Then:

- A set of vertices W is a vertex cover in G if and only if $V \setminus W$ is a clique in $\overline{G}$.

5.15 NP-completeness of Hitting Set

The Hitting Set problem is as follows:

Given a collection C of subsets of a set S and a positive integer k , does there exist a hitting set for C of size at most k ?

A hitting set is a subset $S' \subseteq S$ such that S' contains at least one element from each subset from C .

Hitting Set is in **NP**:

- the subset S' is the certificate
- it can be verified in polynomially time if S' contains an element from each subset in C

To show that Hitting Set is **NP**-complete:

- restrict it to instance with $|c| = 2$ for all $c \in C$
- i.e., every set c in C has 2 elements

Then this restricted problem is equivalent to Vertex Cover:

- let S be the set of vertices of a graph G
- let each subset $c \in C$ correspond to an edge of G
 - if an edge joins vertices u and v , then the corresponding subset is u, v

A hitting set S' must contain at least one element from every subset in C .

Equivalently:

- S' must contain at least one endpoint of every edge in G

which is exactly the definition of a vertex cover.

Therefore:

- Vertex Cover $\leq$ Hitting Set

5.15.1 NP-completeness of 3-SAT

To show that 3-SAT is NP-complete, SAT can be reduced to 3-SAT:

Replace every clause

$$C = x_1 \vee x_2 \vee \dots \vee x_k$$

with $k > 3$ by:

$$C' = (x_1 \vee x_2 \vee y_1) \wedge (\neg y_1 \vee x_3 \vee y_2) \wedge \dots \wedge (\neg y_{k-3} \vee x_{k-1} \vee x_k)$$

C is satisfiable if and only if C' is, since at least one of the literals other than the y 's must be TRUE.

5.16 Reducing SAT to Clique

Another way to show that Clique is **NP**-complete is to reduce SAT to Clique.

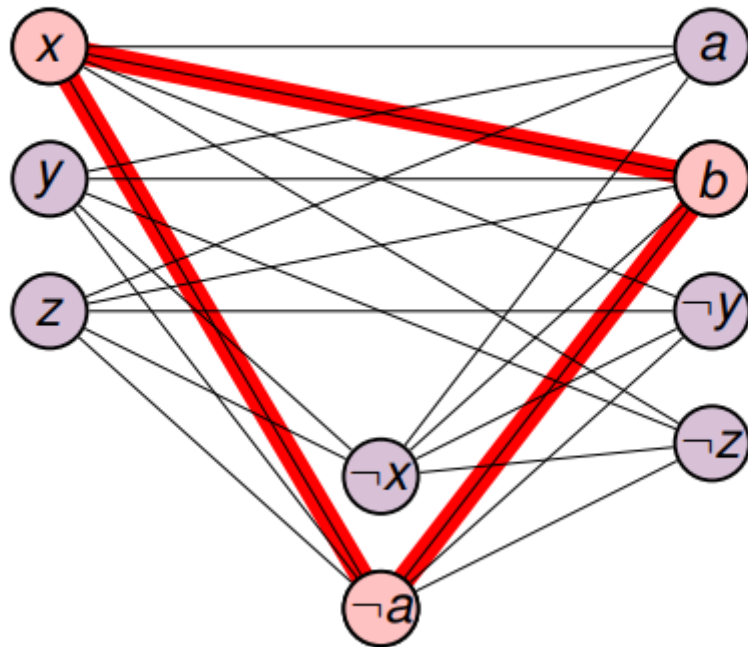
Given a CNF formula f with clauses $C_1, C_2, \dots, C_k$:

- for each literal x in clause C_j , create a vertex
 - each vertex represents a specific occurrence of a literal in a clause
- add an edge between two vertices if:
 - they come from different clauses, and
 - they are not contradictory literals (i.e. not x and $\neg x$)

A clique of size k corresponds to choosing:

- exactly one literal from each clause
- such that all chosen literals are mutually consistent

$$\underbrace{(x \vee y \vee z)}_{C_1} \wedge \underbrace{(\neg x \vee \neg a)}_{C_2} \wedge \underbrace{(a \vee b \vee \neg y \vee \neg z)}_{C_3}$$



$k = 3$

[]: